Overview

A. Machine Learning in Industry

Machine learning has become one of the most important tools used in the tech industry. It powers everything from self-driving cars to virtual assistants. Nearly every company either has machine learning in their tech stack or is planning to incorporate machine learning in the near future.

Many people who have taken a college course or an online course in machine learning are surprised at how different the machine learning process is in industry. Very seldom in industry will people need to know much of the theory or mathematics behind machine learning; rather, the main focus is in applying existing algorithms and models to new problems and datasets.

Furthermore, the majority of work done by machine learning engineers in industry has nothing to do with actually building out machine learning models. Rather, most of the time spent on machine learning projects is in analyzing datasets, creating a data pipeline, and interpreting model results.

The purpose of case studies are to provide learners the chance to work on an actual use case in industry. In this case study we will walk through the industry machine learning process step by step: from initial data preprocessing of raw data all the way to simulated presentation for key decisionmakers.

B. What this course will provide

After taking this course, you'll have completed a project akin to those assigned to industry machine learning engineers. Specifically, you will conduct the following:

- Process and analyze a real industry dataset.
- Understand the machine learning business problem and define a modeling strategy to solve it based on data analysis.

modeling strategy to solve it based on data analysis.

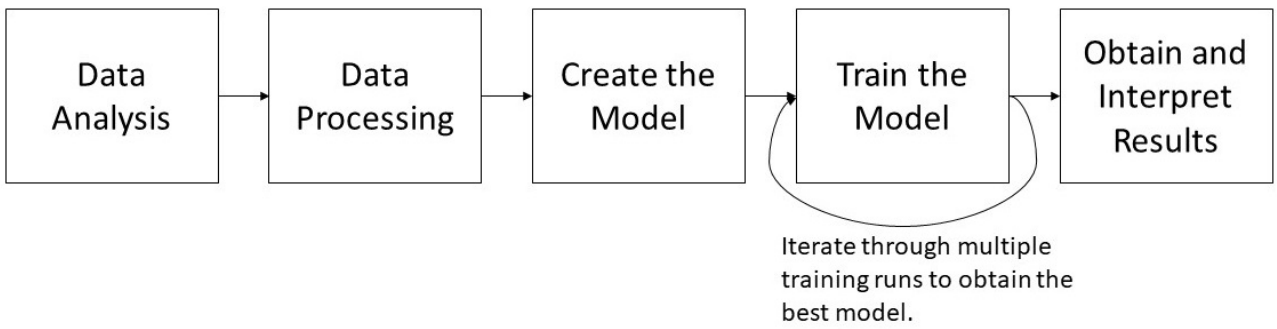
- Create an efficient data pipeline and develop a robust machine learning model for the regression task.
- Interpret model results with respect to the project domain and present the results in a business setting.

Introduction

In this case study, you will be completing a real industry machine learning project. The project involves making sales predictions (sales forecasting) for various stores of a large retail corporation, based on a variety of different features. Along the way, you will also learn techniques for efficient data processing and model training.

A. Industry ML projects

Machine learning projects in industry are comprised of five main parts: data analysis, data processing, creating a model, training the model, and interpreting results. Data analysis is used to confirm that machine learning can actually find useful trends in the dataset to make the requisite predictions. Oftentimes, people will forget to perform data analysis and end up confused why their machine learning model doesn't work well, when in reality it is because the dataset is not conducive to making predictions.



The timeline for an industry ML project.

After data analysis and data processing, we create the appropriate machine learning model given the application and dataset. In most cases, a regular multi-layer perceptron (MLP) will suffice, but sometimes we use a convolutional neural network (CNN) for images/videos or a recurrent neural network (RNN) for text-based data. Creating the model is the easiest part of an

network (RNN) for text-based data. Creating the model is the easiest part of an industry ML project, since you just need to reproduce the code for one of these well-known models..

Finally, after training the model on the dataset, we use it on test data to confirm that the model works. The results obtained from using the model on the test data will be interpreted and reported to the project's supervisors. This is where we can confirm to the supervisors that the model indeed works for the given application, and can be deployed to production.

B. Sales forecasting

One of the most common uses of machine learning in industry is to predict retail sales for consumer-based corporations. Our case study will have you mimic the work of an ML engineer at a large retail corporation, where you'll be creating a model to predict the weekly sales of stores, based on a large training dataset.

In this section of the case study, you'll be performing data analysis to get a feel for the dataset and also confirm that there are enough trends in the dataset to predict weekly sales. You'll also perform a bit of data processing to organize the raw dataset, although the majority of the data processing is done in the **Data Pipeline Creation** section.

The Dataset

Learn about the retail dataset used for the project.

Chapter Goals:

- Learn about the retail dataset used in this case study
- Read the separate data files that comprise the dataset

A. Starting off the project

Let's say you're a machine learning engineer at a large retail corporation, and your supervisor just gave you this dataset and said,

"I want a system that can make future sales predictions for these 45 stores. We want to know whether they'll make enough money to justify keeping them. Here's a dataset containing the past sales of these stores."

This is a pretty short and vague description of the project, which is normally the type of description you'd get from a manager or supervisor. Luckily, we can learn more about the project by looking through the dataset.

Industry data usually comes in CSV files, XLSX spreadsheets, JSON data files, or can be accessed from a database using SQL. In this case, our dataset comes in three CSV files: *weekly_sales.csv*, *features.csv*, and *stores.csv*. We'll use the pandas library's `pd.read_csv` function to read each CSV file into a DataFrame.

For more on pandas and data processing, check out the Machine Learning for Software Engineers course on Educative.

B. Understanding the dataset

Your supervisor gave you some basic details about the dataset. The *weekly_sales.csv* file contains rows detailing the sales (in dollars) for the departments of each store in a given week. We use this file to train a machine learning model to make future weekly sales predictions for each store's department.

```
import pandas as pd

train_df = pd.read_csv('weekly_sales.csv')

print(train_df)
```

DataFrame (train_df) containing data from weekly_sales.csv. The DataFrame contains 421570 rows.

After taking a look at the data using pandas, you confirm that the *weekly_sales.csv* file does indeed match your supervisor's description. There's also an additional column called `'Holiday'`, which is `True` if the row's week has a holiday, otherwise it's `False`.

The *features.csv* file contains potentially useful features, with values given on a weekly basis for each store. These features include a given week's national unemployment rate and the temperature of the region that the store is located in. The *stores.csv* file contains information about each of the 45 stores, specifically the type of store and the size of the store.

We'll take a deeper look at the features and stores CSV files in later chapters.

Time to Code!

In this chapter you'll be completing the `read_dataframes` function, which returns a DataFrame for each of the CSV files in the dataset: *weekly_sales.csv*, *features.csv*, and *stores.csv*.

We can use pandas to read the CSV data into DataFrame objects, then return the three DataFrames.

Set `train_df` equal to `pd.read_csv` applied with `'weekly_sales.csv'`.

Set `features_df` equal to `pd.read_csv` applied with `'features.csv'`.

Set `stores_df` equal to `pd.read_csv` applied with `'stores.csv'`.

Return a tuple containing the three created DataFrames, `train_df`, `features_df`, and `stores_df` (in that order).

```
import pandas as pd

# Read the CSV data files into DataFrames

def read_dataframes():
    # CODE HERE
    pass
```

Missing Data

Learn how to merge feature datasets find missing data in features.

Chapter Goals:

- Merge the two DataFrames containing the feature data
- Learn how to identify features with missing data

A. Merging the features

Both the `features_df` and `stores_df` DataFrames contain feature data, i.e. data related to the stores or weeks that correspond to the rows in `train_df`. Remember each feature is a column of the DataFrame, and that each row in a given column is an entry for that feature. Let's take a look at the specific features contained in each DataFrame.

```
general_features = features_df.columns

print(general_features)
print('General Features: {}\n'.format(general_features.tolist()))

store_features = stores_df.columns
print('Store Features: {}'.format(store_features.tolist()))
```

The code above shows the features (i.e. the columns) of the `features_df` and `stores_df` DataFrames. The `tolist` function converts the features from an `Index` to a list.

You'll notice that both these DataFrames share the `'Store'` feature, which is just the ID of the store for a given row in the DataFrame. Since both these DataFrames contain useful data, we can make things easier for ourselves by merging the two DataFrames into one.

We do this by using the `merge` function and merging the DataFrames based on

the `'Store'` feature.

```
merged_features = features_df.merge(stores_df, on='Store')

print(merged_features)
```

The code above merges the two DataFrames. The new DataFrame (`merged_features`) contains all the features from `features_df` and `stores_df`. It has a total of 8190 rows.

B. Finding missing data

Using the newly merged DataFrame, we can figure out which features contain missing data. This is a crucial step in the data analysis, since we need to perform data processing on any features that have missing row values (i.e. only some of the rows will have entries for that feature).

The pandas library will represent missing values with an `NA` in the DataFrame. We use the `pd.isna` function combined with the `any` function to check which columns contain missing values.

```
na_values = pd.isna(merged_features) # Boolean DataFrame
na_features = na_values.any() # Boolean Series
print(na_features)
```

The `'CPI'` and `'Unemployment'` features contain missing values, along with each `'Markdown'` feature. We'll discuss how to handle these missing values in the next chapter.

Dropping Features

Drop features from the dataset that have too many missing data values.

Chapter Goals:

- Figure out exactly how many missing values are in each feature
- Drop the features that contain too many missing values

A. Counting the missing values

In the previous chapter, we figured out that each of the `'MarkDown'` features, along with the `'CPI'` and `'Unemployment'` features contained missing values. We now want to figure out how many missing values each of these features has, i.e. how many rows of the combined feature DataFrame don't contain a value for the particular feature.

This can be done by counting the number of `True` values for each feature's column in the boolean DataFrame.

```
print(len(na_values))
print(sum(na_values['MarkDown1']))
print(sum(na_values['CPI']))
```

Counting the number of missing values in the 'MarkDown1' and 'CPI' features. The na_values boolean DataFrame represents the locations of missing data.

Since each feature's column contains `True` (equivalent to 1) or `False` (equivalent to 0), we just take the column's sum to count the number of `True`, i.e. missing values.

B. Dropping unusable features

The number of missing values in the `'MarkDown'` features are 4158, 5269, 4577, 4726, and 4140 respectively. Since each of the `'MarkDown'` feature values is

missing in over half DataFrame's rows, we'll consider these features unusable and therefore drop them from the dataset.

```
markdowns = [
    'MarkDown1',
    'MarkDown2',
    'MarkDown3',
    'MarkDown4',
    'MarkDown5'
]
merged_features = merged_features.drop(columns=markdowns)
print(merged_features.columns.tolist())
```

Dropping the 'MarkDown' features from the DataFrame containing all the features.

Both the `'CPI'` and `'Unemployment'` features contain only 585 missing values. This is significantly less than the total number of rows in the DataFrame (8190), so we can still use these features. We'll discuss how to deal with the missing values in the next chapter.

Filling In Data

Fill in missing data for features that only have a few missing values.

Chapter Goals:

- Find the rows that contain missing values for `'CPI'` and `'Unemployment'`
- Fill in the missing values using previous row values

A. Finding the missing values

We previously noted that both the `'CPI'` and `'Unemployment'` features contain 585 missing values. We'll find the row indexes containing these missing values by first converting the feature columns in the `na_values` boolean DataFrame to integers, i.e. 0 and 1.

We then use the `nonzero` function to find the locations of the 1's, which correspond to the `True` values.

```
import numpy as np  # NumPy library

na_cpi_int = na_values['CPI'].astype(int)
na_indexes_cpi = na_cpi_int.to_numpy().nonzero()[0]
na_une_int = na_values['Unemployment'].astype(int)
na_indexes_une = na_une_int.to_numpy().nonzero()[0]

print(np.array_equal(na_indexes_cpi, na_indexes_une))
```

Obtaining the row indexes of the missing values in 'CPI' and 'Unemployment'. The na_values DataFrame is the same boolean DataFrame used in previous chapters.

The row indexes are located in the `na_indexes_cpi` and `na_indexes_une` NumPy arrays, which you can see contain the exact same row indexes (sorted in ascending order). Now let's take a closer look at the exact rows that contain the missing values.

```
na_indexes = na_indexes_cpi
na_rows = merged_features.iloc[na_indexes]
print(na_rows['Date'].unique())  # missing value weeks


print(merged_features['Date'].unique()[-13:])  # final 13 weeks

print(na_rows.groupby('Store').count()['Date'].unique())
```

Finding the unique weeks corresponding to the missing values. The na_rows DataFrame consists of all the rows containing the missing values.

In the code above, we found that the missing values for `'CPI'` and `'Unemployment'` correspond to only 13 weeks, which are in fact the final 13 weeks of the entire dataset. Furthermore, the final line of code shows that each store contains 13 weeks with missing values.

Since there are 45 stores, and 13 weeks missing per store, that gives us the total of 585 rows with missing values.

B. Filling in the values

The `'CPI'` and `'Unemployment'` features correspond to the national consumer price index and unemployment rate. These values have minimal change on a week-to-week basis, so we can fill in the missing values using the final `'CPI'` and `'Unemployment'` values for the week of `'2018-04-26'` (the final week without a missing value).

```
print(na_indexes[0])  # first missing value row index
print()

first_missing_row = merged_features.iloc[169]
print(first_missing_row[['Date','CPI','Unemployment']])
print()

final_val_row = merged_features.iloc[168]
print(final_val_row[['Date','CPI','Unemployment']])
print()

cpi_final_val = merged_features.at[168, 'CPI']
une_final_val = merged_features.at[168, 'Unemployment']
merged_features.at[169, 'CPI'] = cpi_final_val
merged_features.at[169, 'Unemployment'] = une_final_val

new_row = merged_features.iloc[169]
print(new_row[['Date','CPI','Unemployment']])
print()
```

Filling in the values for the first row containing missing values.

Since the row indexes in `na_indexes` are sorted in ascending order, we can fill in the missing values using a `for` loop through `na_indexes` (more details in the coding exercise at the end of this chapter).

C. Imputation variants

Filling in missing data with substituted values is known as imputation. The method we used for our dataset replaced missing values with values from closely related observations. However, there are many other forms of imputation, such as replacing the feature's missing data with the feature's mean, median, or mode.

There are also more advanced imputation methods such as k-Nearest Neighbors (filling in missing values based on similarity scores from the kNN algorithm) and MICE (applying multiple chained imputations, assuming the missing values are randomly distributed across observations).

In most industry cases these advanced methods are not required, since the data is either perfectly cleaned or the missing values are scarce. Nevertheless, the advanced methods could be useful when dealing with open source datasets, since these tend to be more incomplete.

Time to Code!

In this chapter you'll be completing the `impute_data` function. You'll use final non-missing value to fill in the missing feature values for `'CPI'` and `'Unemployment'` (as demonstrated in the chapter example).

The `na_indexes_cpi` object represents an array containing the row indexes of the `'CPI'` feature's missing values.

Since these indexes are in ascending sorted order, we can iterate through them and set the missing values equal to the previous row's value in `merged_features`.

Create a `for` loop that iterates through `na_indexes_cpi` using variable `i`.

To ensure that we're updating the actual `merged_features` DataFrame, and not just a copy of the `'CPI'` column, we need to use the `at` function.

Inside the `for` loop, set the value at row `i` in the `'CPI'` column of `merged_features` equal to the value at row `i - 1` in the `'CPI'` column. Make sure to use the `at` function for both the setting and getting operations.

We'll now perform the same steps to fill in the missing data for the `'Unemployment'` column.

Repeat the same process for the `'Unemployment'` column using the `na_indexes_une` array.

```
import pandas as pd

# Fill in missing data
def impute_data(merged_features, na_indexes_cpi, na_indexes_une):
    # CODE HERE
    pass
```

Merging Data

Merge the main training dataset with its corresponding feature data.

Chapter Goals:

- Create the final dataset by merging the training and features DataFrames

A. The final dataset

For the same organizational reasons we had in merging the features and stores DataFrames, we'll now merge the training and combined features DataFrames.

Remember that the stores DataFrame contains potentially useful features listed weekly by store, and the stores DataFrame contains the type and size of each store.

```
train_df = pd.read_csv('weekly_sales.csv')
print(train_df.columns.tolist())

# Merged and imputed stores + features
print(merged_features.columns.tolist())
```

Viewing the columns of the train_df and merged_features DataFrames.

The code above shows that the two DataFrames share the features `'Store'`, `'Date'`, and `'IsHoliday'`. Therefore, we merge the DataFrames on these three features.

While the `'Date'` feature is useful in the sense that it allows us to identify important values for a given week, like unemployment rate or CPI, it's not used directly in training a machine learning model. Therefore, we drop it from the final dataset.

```
features = ['Store', 'Date', 'IsHoliday']
final_dataset = train_df.merge(merged_features, on=features)
final_dataset = final_dataset.drop(columns=['Date'])


print(final_dataset.columns.tolist())
```

Creating the final dataset.

Categorical Data

Learn about categorical data and how it is used in a dataset.

Chapter Goals:

- Analyze the categorical features in the dataset

A. The dataset format

When using the dataset to train a machine learning model, each feature needs to be an integer, float, or string type. The float data is the numeric data, i.e. the data that can be quantified and analyzed using operations like mean or standard deviation. The string data is categorical, meaning that each string represents some unique category for the feature. Integer data can be either numeric (e.g. kilometer distance) or categorical (e.g. year of birth).

In the final dataset we're using, the categorical features are the `'Store'`, `'Type'`, and `'Dept'` features. Since we already know there are 45 stores, labeled from 1 to 45, we just need to investigate the `'Type'` and `'Dept'` features.

```
print(final_dataset['Type'].unique())
print(final_dataset['Dept'].unique())
```

There are only three categories of stores shown in the `'Type'` feature: `'A'`, `'B'`, and `'C'`. There are 81 store departments, and each is a positive integer less than 100. We'll discuss how to process and use categorical and numeric features in a machine learning model later in this course.

Time to Code!

When using the dataset for training a machine learning model, we want the features to either be integer, float, or string type. Currently the only feature in

`final_dataset` that is not one of those types is `'IsHoliday'`. Therefore, the

coding exercise for this chapter is to convert the `'IsHoliday'` boolean values to binary integers.

Cast the `'IsHoliday'` feature of `final_dataset` to `int` type.

```
import pandas as pd

# Update IsHoliday values
# CODE HERE
```

Scatter Plot

Learn how to create scatter plots comparing numerical features to sales.

Chapter Goals:

- Create a scatter plot to visualize the relationships between dataset features and weekly sales

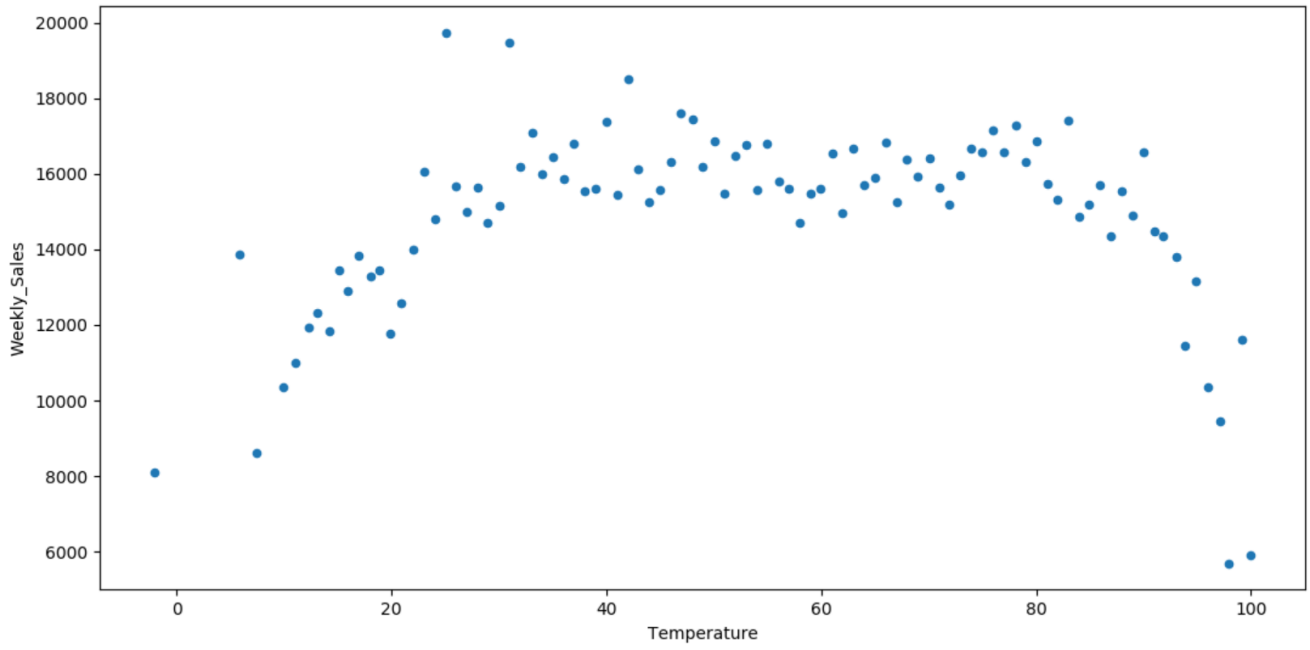
A. Plotting data

The most important part of interpreting data is being able to visualize trends and correlations between dataset features. We do this through data plots, which allows us to easily discover interesting patterns in the dataset and decide whether the dataset is feasible for training a machine learning model.

We create the data plots using the pyplot API of Matplotlib. One of the most common plots for data analysis is the 2-D scatter plot. It's used for plotting the relationship between a numeric dependent feature (Y-axis) and a numeric independent feature (X-axis). Since we're trying to predict weekly sales for stores, we'll make plots with the `'Weekly_Sales'` feature as the dependent feature.

```
import matplotlib.pyplot as plt

plot_df = final_dataset[['Weekly_Sales', 'Temperature']]
rounded_temp = plot_df['Temperature'].round(0)  # nearest integer
plot_df = plot_df.groupby(rounded_temp).mean()
plot_df.plot.scatter(x='Temperature', y='Weekly_Sales')
plt.show()
```



Creating a scatter plot with 'Temperature' as the independent feature and 'Weekly_Sales' as the dependent feature.

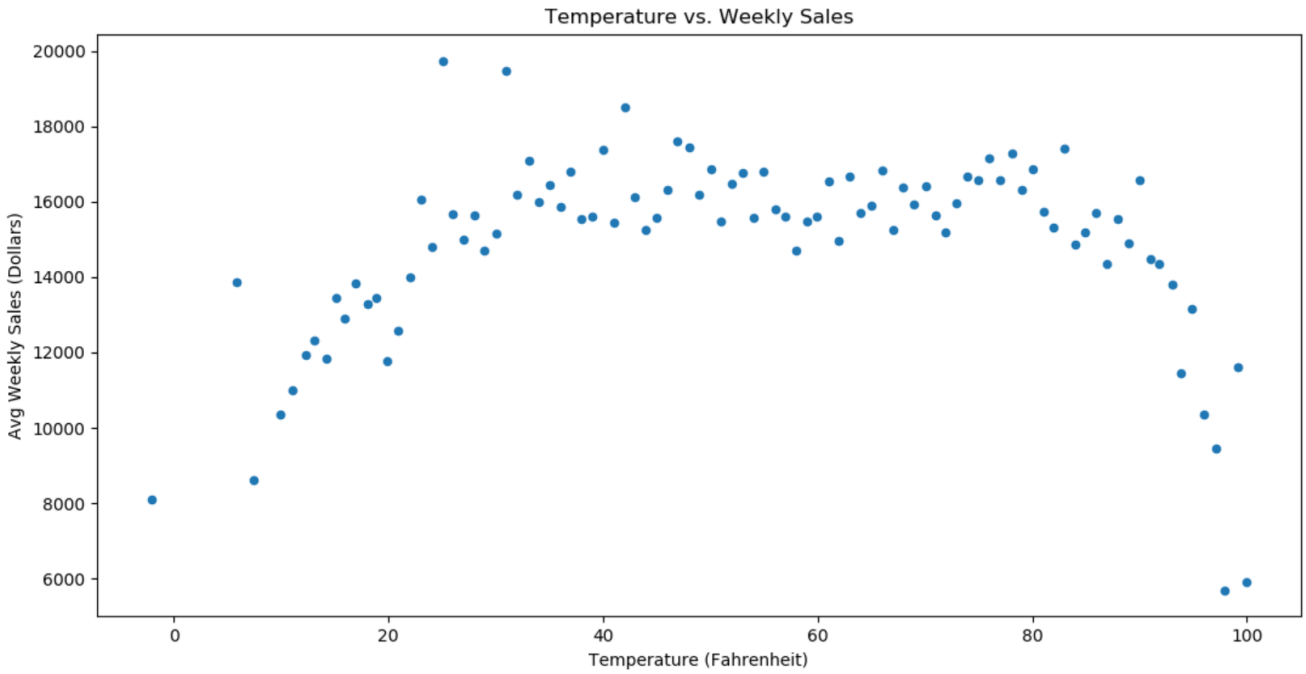
In the code, we rounded the temperature to the nearest integer and took the average weekly sales at each integer temperature value (degrees Fahrenheit).

We'll do more detailed analysis of the feature scatter plots in later chapters, but you can tell from the above scatter plot that sales are pretty consistent around $160,000 weekly, although they tend to drop at the high and low ends of the temperature spectrum.

For organizational purposes, and to present a more professional plot to the project supervisor, we should label and title the scatter plots appropriately.

```
import matplotlib.pyplot as plt

plot_df = final_dataset[['Weekly_Sales', 'Temperature']]
rounded_temp = plot_df['Temperature'].round(0)  # nearest integer
plot_df = plot_df.groupby(rounded_temp).mean()
plot_df.plot.scatter(x='Temperature', y='Weekly_Sales')
plt.title('Temperature vs. Weekly Sales')
plt.xlabel('Temperature (Fahrenheit)')
plt.ylabel('Avg Weekly Sales (Dollars)')
plt.show()
```



Labeling and titling the previous scatter plot. The plot_df variable contains the average weekly sales at each integer temperature value.

Bar Plot

Learn how to create bar plots comparing categorical features to sales.

Chapter Goals:

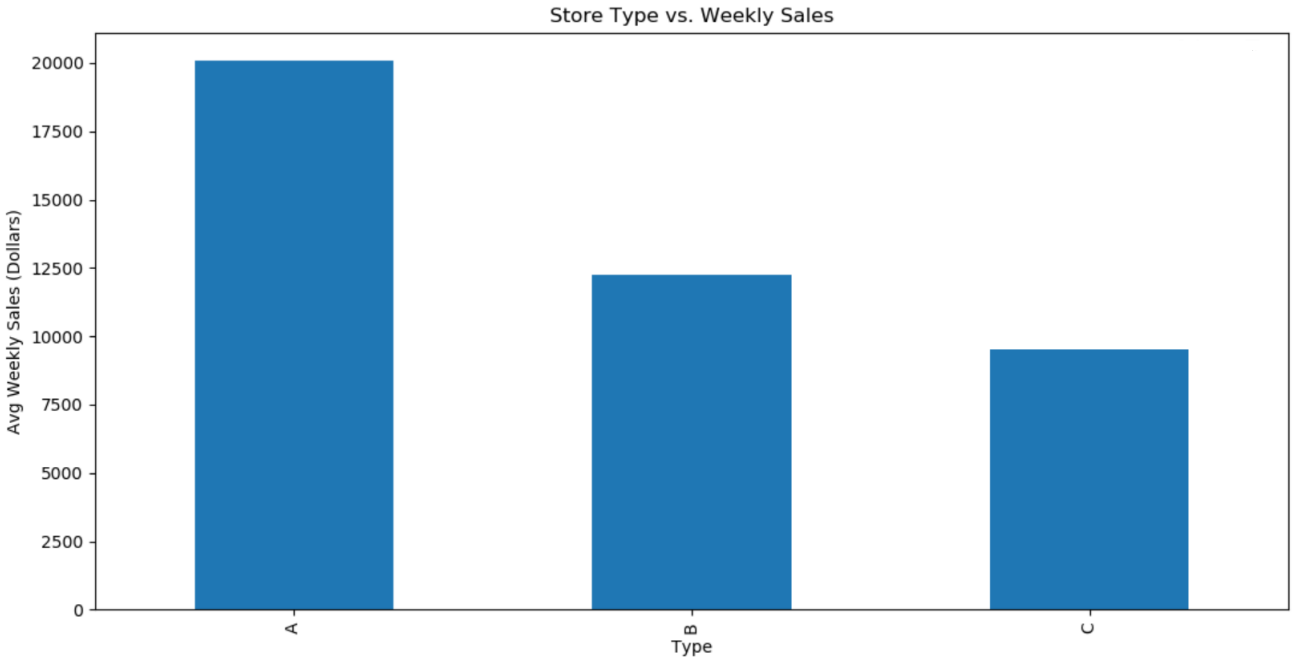
- Create a bar plot to visualize the correlation between dataset features and weekly sales

A. Categorical feature plots

For categorical features, it doesn't make sense to create a scatter plot. Instead, when we're using a categorical feature as the independent variable (with a numeric feature as the dependent variable), we'll make a bar plot.

When using weekly sales as the dependent variable, we'll create bar plots that show the average weekly sale amount for each category in a categorical feature.

```
plot_df = final_dataset[['Weekly_Sales', 'Type']]
plot_df = plot_df.groupby('Type').mean()
plot_df.plot.bar()
plt.title('Store Type vs. Weekly Sales')
plt.xlabel('Type')
plt.ylabel('Avg Weekly Sales (Dollars)')
plt.show()
```



Creating a bar plot with 'Type' as the independent feature and average 'Weekly_Sales' as the dependent feature.

The above bar plot shows that stores of type `A` have significantly higher average weekly sales than stores of type `B` or `C`.

Interpreting Plots

Interpret scatter and bar plots in the context of the project.

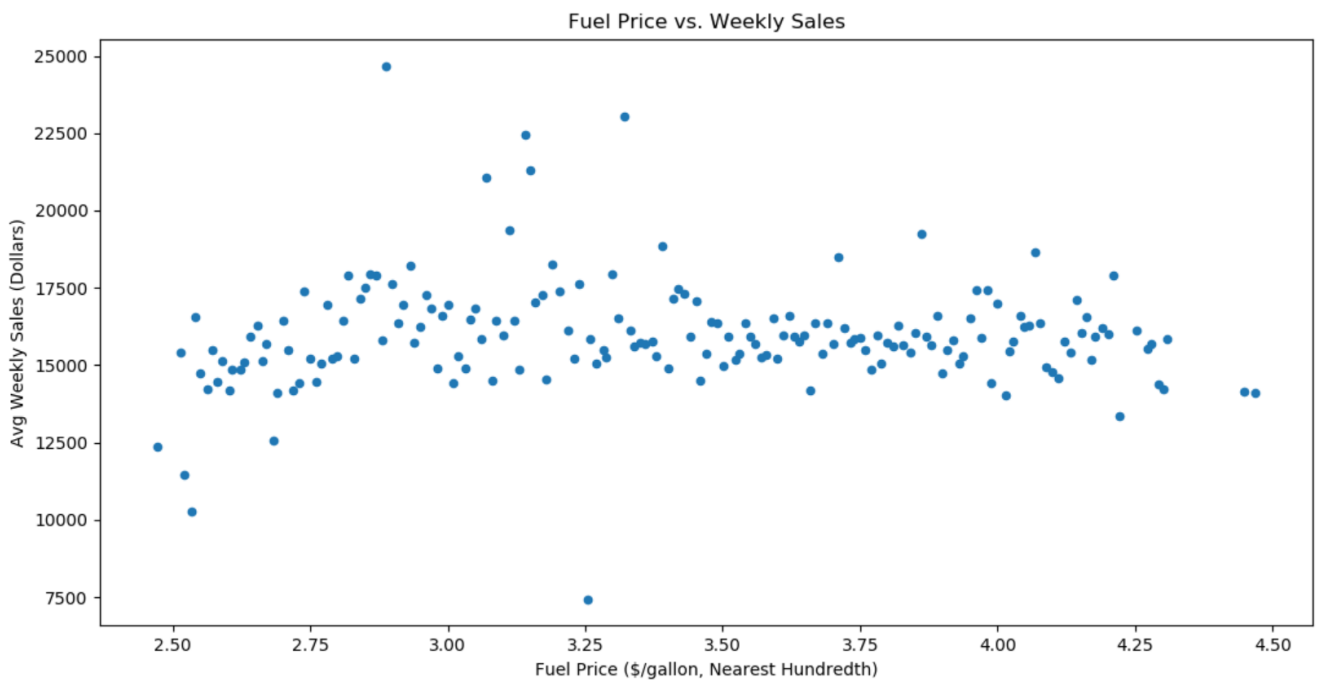
Chapter Goals:

- Analyze plots of the dataset's features with respect to weekly sales and usage in a machine learning model

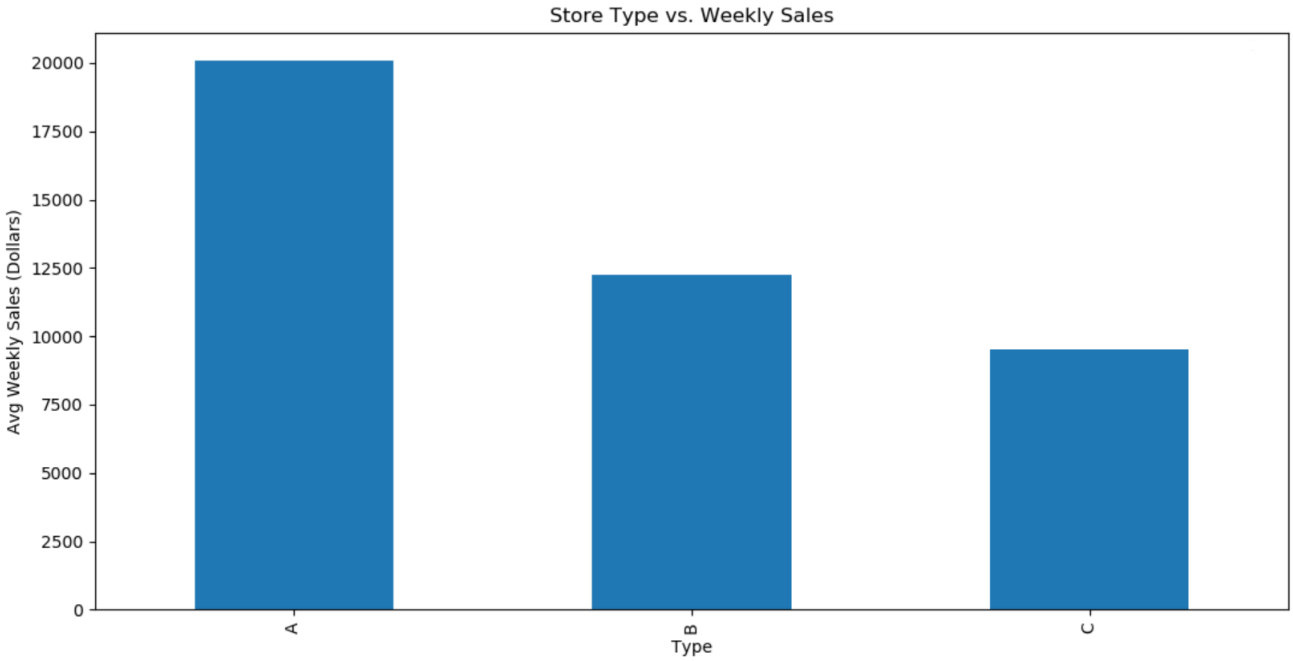
A. Analyzing within context

After creating the dataset plots, we should analyze them to determine whether they're useful in the context of our problem. For us, this means deciding whether there's enough correlation between the dataset features and weekly sales to train a machine learning model.

The main thing we're looking for in the plots is non-uniform distributions. A uniform distribution means that the weekly sales are identical regardless of the data feature's value. A non-uniform distribution, such as a normal distribution or a multi-modal distribution, shows that the dataset feature can potentially be used by a machine learning model to predict the sales.



The scatter plot for the 'Fuel' feature shows a relatively uniform distribution.



The bar plot for the 'Type' feature shows a non-uniform distribution.

It can also be beneficial to plot multiple features at once against the weekly sales. Sometimes, when plotting multiple features, we can find trends that would not have been found from the single feature's plot.

Quiz

1

Why is data analysis important before creating a machine learning model for a task?

COMPLETED 0%

1 of 4

Introduction

In this section of the course you will be performing data processing on the final dataset from the **Preliminary Data Analysis** section. Specifically, you'll be building the input pipeline for training and evaluating the machine learning model.

A. Additional data processing

In the **Preliminary Data Analysis** section, we performed data analysis on the retail dataset and concluded that there is a strong enough correlation from the dataset's features to predict weekly store sales.

The files in that chapter were small enough to use relatively simple techniques. If the files were larger and more resource intensive, we would have used the techniques laid out in the **Efficient Data Processing Techniques** section.

After informing the project supervisor that the prediction task is viable, we begin working on the machine learning model.

However, before writing any actual machine learning code, we know that we need to continue processing the data to create an efficient *input pipeline*. The input pipeline represents how the data will be passed into the model for each step of training or evaluation. Since training the model requires thousands of steps, it is important that the input pipeline is as efficient as possible.

The final dataset we created was stored in a pandas DataFrame. Since the DataFrame is not the most efficient data storage for the input pipeline, we'll need to perform additional processing to create a more efficient solution.

Splitting Datasets

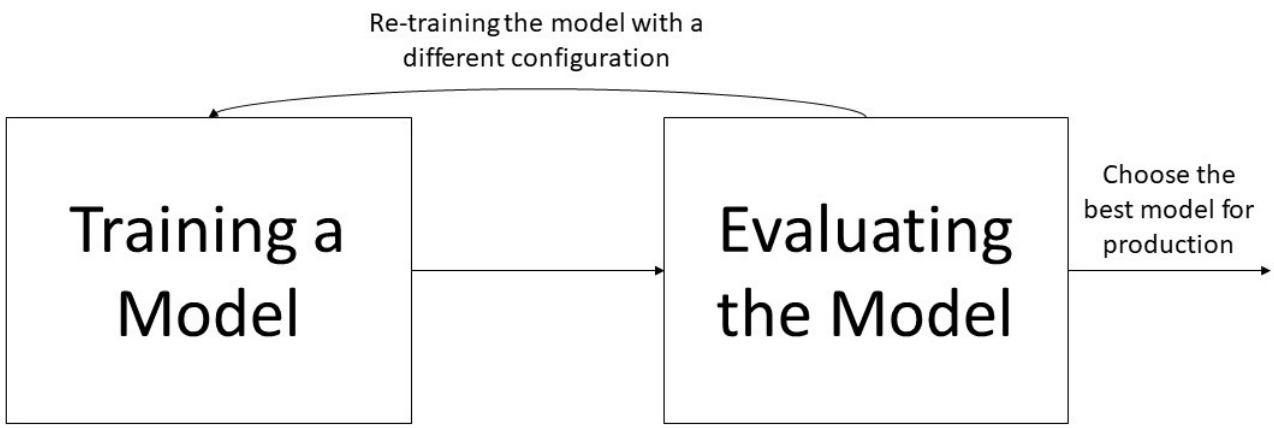
Split the overall project's dataset into training and evaluation sets.

Chapter Goals:

- Learn about training and evaluation sets
- Split the project's final dataset into training and evaluation sets

A. Training and evaluation

There are two main components in creating a machine learning model: training and evaluation. Training is the foundation of machine learning, but evaluation is just as important. Model evaluation gives us a concrete idea of just how good the model is after training, and it allows us to compare the performances for different configurations of the model.



The process for training and evaluating a machine learning model.

B. Set proportions

The question now becomes how much of the data we use for training and how much we use for evaluation. We should use a lot more data for training (the training set) compared to the data for evaluation (the evaluation set). The exact amount is up to the machine learning engineer to decide.

Using more data in training would potentially improve the model's

performance, but it would limit us in how accurate our evaluation is due to the limited evaluation set size. On the other hand, having a larger evaluation set would give us more confidence in our evaluation process' accuracy, but it might limit the amount and diversity of the data in training.

In our case study, we choose a 90-10 split, meaning that the training set comprises 90% of the final dataset while the evaluation set comprises 10%. Since the overall dataset is pretty large, a 10% evaluation set still gives us a good representation of the overall dataset. Therefore, we can afford to put 90% of the dataset in training to maximize the model's performance.

C. Removing systematic trends

Before splitting the final dataset into training and evaluation sets, we need to randomly shuffle it. This is because the dataset is currently sorted by date and store, which is a systematic trend that we need to remove.

```
print(final_dataset.columns)
print(final_dataset)
```

The final_dataset object from the Preliminary Data Analysis section, with the 'Date' feature not dropped. It is currently sorted by the 'Date' and 'Store' features.

If we don't remove this trend, each training step will have data that is too similar to adjacent training steps, since they will likely be for adjacent weeks of the same store. This is an artificial characteristic that doesn't appear in real life, so it would negatively impact the model for real life predictions.

```
# Split the final pandas DataFrame into training and evaluation sets
def split_train_eval(final_dataset):
    # CODE HERE
    pass
```

Integer Features

Learn about the integer features used in the dataset.

Chapter Goals:

- Add the integer features of a DataFrame's row to a feature dictionary

A. Using Example objects

Each row of the final pandas DataFrame from the Data Analysis Lab contains the feature data for one data observation, i.e. the feature data for one store's sales in a particular week. To optimize the input pipeline, we want to convert each DataFrame row into a TensorFlow Example object. By using Example objects in the input pipeline, we're able to efficiently feed the data into a machine learning model.

After converting a DataFrame row to a TensorFlow Example, the row's integer valued features will be represented by `Int64List` TensorFlow Feature objects. From the analysis of our dataset, we know that the features with integer values are `'Store'`, `'Dept'`, `'IsHoliday'`, and `'Size'`.

```
print(feature_dict['Size'])
print(feature_dict['Store'])
print(feature_dict['Dept'])
print(feature_dict['IsHoliday'])
```

For a single row in the pandas DataFrame (i.e. a single observation in the dataset), this code prints out the TensorFlow Feature objects for each of the integer features in that row. The feature_dict dictionary maps string names to Feature objects.

Time to Code!

In this chapter you'll be completing the `add_int_features` function, which adds all the integer features in a dataset row to the feature dictionary.

The features that contain integer values are: `'Store'`, `'Dept'`, `'IsHoliday'`, and `'Size'`.

Set `int_vals` equal to a list containing the feature names with integer values.

For each integer valued feature, we'll create an `Int64List` containing the feature's value from `dataset_row`.

Create a `for` loop that iterates through `int_vals` using a variable named `feature_name`.
Inside the `for` loop, set `list_val` equal to `tf.train.Int64List` initialized with the `value` keyword argument set to a singleton list containing `dataset_row[feature_name]`.

We can now map the feature's name to a TensorFlow Feature object representing its integer value.

Inside the `for` loop, set `feature_name` as a key in `feature_dict`. The value it maps to will be `tf.train.Feature` initialized with the `int64_list` keyword argument set to `list_val`.

```
import tensorflow as tf

# Add the integer Feature objects to the feature dictionary
def add_int_features(dataset_row, feature_dict):
    # CODE HERE
    pass
```

Float Features

Learn about the float features used in the dataset.

Chapter Goals:

- Add the float features of a DataFrame's row to a feature dictionary

A. Adding float features

Similar to the integer valued features in the DataFrame row, we also convert the row's float valued features when creating the Example object. Each of the float features will be represented by `FloatList` TensorFlow Feature objects after the row is converted to an Example.

From the analysis of our dataset, we know that the float valued features are `'Temperature'`, `'FuelPrice'`, `'CPI'`, `'Unemployment'`, and `'Weekly_Sales'`.

```
print(feature_dict['Temperature'])
print(feature_dict['Fuel_Price'])
print(feature_dict['CPI'])
print(feature_dict['Unemployment'])
print(feature_dict['Weekly_Sales'])
```

The TensorFlow Feature objects for each of the float features in a single observation in the dataset (i.e. a row in the pandas DataFrame). The feature_dict dictionary maps string names to Feature objects.

The `'Weekly_Sales'` feature acts as the *label* used in training and evaluating the machine learning model. However, this means that the feature is not present outside of training/evaluation, i.e. when making real-life sales predictions with the model.

Therefore, we only include the `'Weekly_Sales'` feature in our Example objects for training and evaluation. In other words, only when the Example object should contain a label.

Time to Code!

In this chapter you'll be completing the `add_float_features` function, which adds all the float features in a dataset row to the feature dictionary.

The features that contain float values are: `'Temperature'`, `'Fuel_Price'`, `'CPI'`, `'Unemployment'`, and `'Weekly_Sales'`.

However, we only use the `'Weekly_Sales'` feature if `has_labels` is `True`. This is because the `'Weekly_Sales'` feature represents the *label* used in training/evaluating the machine learning model, which is not present when making predictions.

Set `float_vals` equal to a list containing the feature names with float values. Only include `'Weekly_Sales'` in the list if `has_labels` is `True`.

For each float valued feature, we'll create a `FloatList` containing the feature's value from `dataset_row`.

Create a `for` loop that iterates through `float_vals` using a variable named `feature_name`.
Inside the `for` loop, set `list_val` equal to `tf.train.FloatList` initialized with the `value` keyword argument set to a singleton list containing `dataset_row[feature_name]`.

We can now map the feature's name to a TensorFlow Feature object representing its float value.

Inside the `for` loop, set `feature_name` as a key in `feature_dict`. The value it maps to will be `tf.train.Feature` initialized with the `float_list` keyword argument set to `list_val`.

```
import tensorflow as tf

# Add the float Feature objects to the feature dictionary
def add_float_features(dataset_row, feature_dict, has_labels):
    # CODE HERE
    pass
```

String Features

Learn about the string features used in the dataset.

Chapter Goals:

- Add the string features of a DataFrame's row to a feature dictionary

A. Adding string features

The third type of TensorFlow Feature object that can be used in an Example object is a `BytesList` TensorFlow Feature. This can represent either byte values (e.g. image data) or string values. For string values, we need to convert them to the `bytes` type prior to initializing a `BytesList` Feature object.

```
s = 'hello world'
byte_s = s.encode()  # byte string
bytes_list = tf.train.BytesList(value=[byte_s])
feature = tf.train.Feature(bytes_list=bytes_list)
print(feature)
```

Creating a BytesList TensorFlow Feature from a string. The encode function converts the string to bytes type.

From the analysis of our dataset, we know that the only feature containing string values is `'Type'`.

Time to Code!

In this chapter you'll be completing the `create_example` function, which creates an Example object from a row of the dataset.

We've already initialized a feature dictionary and added the integer and float features, using the functions from the previous two chapters. The only feature that contains string values is `'Type'`, so we need to convert the `'Type'` value in `dataset_row` to `bytes`, and then create a `BytesList`.

Set `byte_type` equal to `dataset_row['Type']`, converted to `bytes`.
Set `list_val` equal to `tf.train.BytesList` initialized with the `value` keyword argument set to a singleton list containing `byte_type`.

We can now complete the `feature_dict` by mapping `'Type'` to its corresponding TensorFlow Feature object.

Put `'Type'` as a key in `feature_dict`, and map it to a `tf.train.Feature` object initialized with the `bytes_list` keyword argument set to `list_val`.

Using the completed `feature_dict` dictionary, we'll create and return a TensorFlow Example object containing the values in `dataset_row`.

Set `features_obj` equal to `tf.train.Features` initialized with the `feature` keyword argument set to `feature_dict`.

Return a `tf.train.Example` object initialized with the `features` keyword argument set to `features_obj`.

```
import tensorflow as tf

# Create an Example object from a pandas DataFrame row
def create_example(dataset_row, has_labels):
    feature_dict = {}
    add_int_features(dataset_row, feature_dict)
    add_float_features(dataset_row, feature_dict, has_labels)
    # CODE HERE
```

Writing TFRecords

Write serialized Example objects into TFRecords files.

Chapter Goals:

- Write the training and evaluation set data into TFRecords files

A. Writing Example data

Now that we've completed the function to convert each DataFrame row into an Example object, we can create the efficient input pipeline storage for both the training and evaluation sets. The data storage will be in the form of TFRecords files, which hold serialized Example objects.

The `write_tfrecords` function (shown below) writes the data from a given DataFrame into a TFRecords file. It uses the `create_example` function from the previous chapter to convert each row of the dataset into an Example object. Each Example object is then serialized and written into the TFRecords file.

```
import tensorflow as tf

# Write serialized Example objects to a TFRecords file
def write_tfrecords(dataset, has_labels, tfrecords_file):
    writer = tf.python_io.TFRecordWriter(tfrecords_file)
    for i in range(len(dataset)):
        example = create_example(dataset.iloc[i], has_labels)
        writer.write(example.SerializeToString())
    writer.close()
```

We can use the above function to write the training set's serialized Example data into a TFRecords file called *train.tfrecords* and the evaluation set's serialized Example data into a TFRecords file called *eval.tfrecords*. These files will then be used in the input pipeline for the machine learning model.

```
# train_set is the training DataFrame
write_tfrecords(train_set, 'train.tfrecords')

# eval_set is the evaluation DataFrame
write_tfrecords(eval_set, 'eval.tfrecords')
```

Example Spec

Create an Example spec used for parsing serialized Examples.

Chapter Goals:

- Create the Example spec that's used to parse serialized Example objects

A. Creating the Example spec

Since the data is now stored as serialized Example objects in TFRecords files, we need to create an Example spec, which allows us to parse the serialized Examples in the input pipeline. The Example spec gives specifications on each of the dataset's features, specifically the shape and type of the feature's values.

The Example spec is just a Python dictionary, mapping feature names to `FixedLenFeature` objects. For our dataset, each of the `FixedLenFeature` objects has shape `()`. This is because each feature contains a single value per data observation, i.e. each feature of our dataset's Example objects contains one value (as opposed to an array of values).

The type of each `FixedLenFeature` depends on the type of values in the feature. Integer valued features will have `tf.int64` type, float valued features will have `tf.float32` type, and string valued features (in our case, only the `'Type'` feature) will have `tf.string` type.

```
import tensorflow as tf

example_spec = {}
example_spec['Store'] = tf.FixedLenFeature((), tf.int64)
example_spec['CPI'] = tf.FixedLenFeature((), tf.float32)
example_spec['Type'] = tf.FixedLenFeature((), tf.string)
```

Creating the Example spec for our dataset. The above code's example_spec, which is incomplete, contains specifications for the 'Store', 'CPI', and 'Type' features.

Time to Code!

In this chapter you'll be completing the `create_example_spec` function, which creates an Example spec based on the dataset features. The function is already filled with code that provides the feature names for the dataset.

For each of the integer features, we'll create a `FixedLenFeature` object with shape `()` and type `tf.int64`.

Create a `for` loop that iterates through `int_vals` using a variable named `feature_name`.
Inside the `for` loop, set `feature_name` as a key in `example_spec`. The value it maps to will be `tf.FixedLenFeature` initialized with the specified shape and type.

We'll now repeat the previous step for each of the float features. The shape of the `FixedLenFeature` objects is still `()`, but now the type is `tf.float32`.

Create another `for` loop (outside the previous one) that iterates through `float_vals` using a variable named `feature_name`.
Inside the `for` loop, set `feature_name` as a key in `example_spec`. The value it maps to will be `tf.FixedLenFeature` initialized with the specified shape and type.

Finally, we'll create a `FixedLenFeature` for the only string feature (`'Type'`), which will have shape `()` and type `tf.string`.

Outside the `for` loops, set `example_spec['Type']` to a `tf.FixedLenFeature` object initialized with the specified shape and type. Then return `example_spec`.

```
import tensorflow as tf

# Create the spec used when parsing the Example object
def create_example_spec(has_labels):
    example_spec = {}
    int_vals = ['Store', 'Dept', 'IsHoliday', 'Size']
    float_vals = ['Temperature', 'Fuel_Price', 'CPI', 'Unemployment']
    if has_labels:
        float_vals.append('Weekly_Sales')
    # CODE HERE
```

Parsing Examples

Set up a function for parsing Example objects with and without labels.

Chapter Goals:

- Create a function to parse feature data from serialized Example objects

A. Extracting feature data

The input pipeline consists of reading serialized Example objects from the TFRecords file and parsing feature data from the serialized Examples. We parse feature data for a single Example (which represents data for one DataFrame row) using the `tf.parse_single_example` function.

```
import tensorflow as tf

def create_example_spec(has_labels):
    example_spec = {}
    int_vals = ['Store', 'Dept', 'IsHoliday', 'Size']
    float_vals = ['Temperature', 'Fuel_Price', 'CPI', 'Unemployment']
    if has_labels:
        float_vals.append('Weekly_Sales')
    for feature_name in int_vals:
        example_spec[feature_name] = tf.FixedLenFeature((), tf.int64)
    for feature_name in float_vals:
        example_spec[feature_name] = tf.FixedLenFeature((), tf.float32)
    example_spec['Type'] = tf.FixedLenFeature((), tf.string)
    return example_spec

example_spec = create_example_spec(True)

parsed_example = tf.parse_single_example(ser_ex, example_spec)
print(parsed_example)
```

Parsing feature data from a serialized Example (ser_ex) using its corresponding Example spec (example_spec).

B. Labeled data

For both training and evaluation, we require the data to be labeled in order to

calculate the loss for our machine learning model. Since our model is trained

to predict weekly sales, we use the `'Weekly_Sales'` feature as the label for each data observation.

This means that every feature apart from `'Weekly_Sales'` is used as input for the model, which then predicts a `'Weekly_Sales'` value based on these input features. The value is compared to the actual `'Weekly_Sales'` label to assess the model's performance.

Time to Code!

In this chapter you'll be completing the `parse_feature` function, which parses features from a serialized Example object.

We first use the input Example spec argument, `example_spec`, to parse a dictionary of features from the input serialized Example, `ser_ex`.

Set `parsed_features` equal to `tf.parse_single_example` with `ser_ex` and `example_spec` as the required arguments.

The `'Weekly_Sales'` feature is not actually used as an input for the machine learning model. Instead, it is used as a label during training and evaluation.

Therefore, we'll separate the `'Weekly_Sales'` feature from the rest of the features (if it's even present).

Create a dictionary called `features` which is identical to `parsed_features`, but does not contain the `'Weekly_Sales'` key.

If `has_labels` is `False` (meaning the `'Weekly_Sales'` feature is not present), we'll just return the rest of the features.

Create an `if` block that checks if `has_labels` is `False`. Inside the `if` block, return `features`.

If `has_labels` is `True`, we can extract the `'Weekly_Sales'` feature from the parsed features, which is used as a label.

Outside the `if` block, set `label` equal to the value in the `parsed_features` dictionary corresponding to `'Weekly_Sales'`. Return the tuple `features, label`.

```
import tensorflow as tf

# Helper function to convert serialized Example objects into features
def parse_features(ser_ex, example_spec, has_labels):
    # CODE HERE
    pass
```

TFRecords Dataset

Create a TFRecords dataset configured specifically for the project's dataset.

Chapter Goals:

- Create a TFRecords dataset for the input pipeline

A. Dataset from TFRecords file

After setting up the Example spec and feature parsing functions, we're finally ready to create TensorFlow datasets from the TFRecords files for both training and evaluation.

```
import tensorflow as tf

train_file = 'train.tfrecords'
eval_file = 'eval.tfrecords'
train_dataset = tf.data.TFRecordDataset(train_file)
eval_dataset = tf.data.TFRecordDataset(eval_file)
```

Parsing feature data from a serialized Example (ser_ex) using its corresponding Example spec (example_spec).

The TFRecords datasets contain serialized Example objects. Using the Example spec and feature parsing functions, we can convert each serialized Example to a tuple containing the Example's feature data and label.

```
import tensorflow as tf

example_spec = create_example_spec(True)
parse_fn = lambda ser_ex: parse_features(ser_ex, example_spec, True)
train_dataset = train_dataset.map(parse_fn)
eval_dataset = eval_dataset.map(parse_fn)
```

Using the functions from the previous two chapters to modify the TFRecords datasets

Using the functions from the previous two chapters to modify the TFRecords datasets.

The TFRecords dataset's `map` function allows us to apply the parsing function (`parse_fn`) to each serialized Example in the dataset. Since the `parse_features` function takes in two arguments, and `map` can only be used on functions with one argument, we use a single argument lambda function to wrap around `parse_features`.

B. Configuring the dataset

Now that we have the datasets for the input pipeline, we can decide how to configure them. Specifically, configuration refers to the shuffling, repetition, and batch size for the dataset.

For a refresher, recall the Configuration chapter from the previous section.

Shuffling datasets is always a good idea for training and evaluation, since it randomizes the order in which the data is passed into the machine learning model. The type of randomness depends on the buffer size for the shuffling.

We'll apply uniform random shuffling, which means that the buffer size needs to be at least the size of the dataset. In the **Preliminary Data Analysis** section, we calculated that the entire feature dataset contained 421570 rows. Therefore, we'll use a buffer size of 421570, which is guaranteed to be larger than the training and evaluation sets.

```
import tensorflow as tf

train_dataset = train_dataset.shuffle(421570)
eval_dataset = eval_dataset.shuffle(421570)
```

Uniformly randomly shuffling the datasets. Note that this is only applied during training/evaluation (i.e. when the dataset contains labels).

We also want to run training indefinitely, until we decide to kill the model running process manually (i.e. with `CTRL+C` or `CMD+C`). Evaluation is done with a single run-through of the dataset.

```
import tensorflow as tf
```

```
train_dataset = train_dataset.repeat()
```

Repeating the datasets indefinitely. The training will run until we manually kill the process.

Finally, we can set the dataset batch sizes, so that each training/evaluation step contains multiple data observations. Given the different sizes of the training and evaluation sets, we'll use batch sizes of 100 and 20, respectively.

```
import tensorflow as tf

train_dataset = train_dataset.batch(100)
eval_dataset = eval_dataset.batch(20)
```

Setting the batch size for the datasets.

Numeric Columns

Learn about the numeric feature columns for the ML model's input layer.

Chapter Goals:

- Process the numeric feature columns used for the machine learning model's input layer

A. Model input layer

Our dataset now contains the feature data and label for each observation. In the **Model Creation** section, we'll convert the feature data into an input layer for the machine learning model. To do that, we first need to set up the necessary feature columns.

Similar to how the Example spec provided specifications for how to parse each Example object, the feature columns give specifications for how the feature data can be aggregated into an input layer. There are three types of feature columns that we'll use: numeric columns, indicator columns, and embedding columns.

B. Numeric data

Numeric feature columns are used for the numeric data in our dataset, i.e. the quantifiable data. There are five features that contain numeric data: `'Size'`, `'Temperature'`, `'Fuel_Price'`, `'CPI'`, and `'Unemployment'`. We use the same process to create numeric feature columns for each of these features.

```
import tensorflow as tf

def add_numeric_columns(feature_columns):
    numeric_features = ['Size', 'Temperature', 'Fuel_Price', 'CPI', 'Unemployment']
    for feature_name in numeric_features:
        feature_col = tf.feature_column.numeric_column(feature_name, shape=())
        feature_columns.append(feature_col)

# Add the numeric feature columns to the list of dataset feature columns
dataset_feature_columns = []
add_numeric_columns(dataset_feature_columns)
```

```
print(dataset_feature_columns)
```

Function that creates the numeric feature columns for the dataset.

Indicator Columns

Learn about the indicator feature columns for the ML model's input layer.

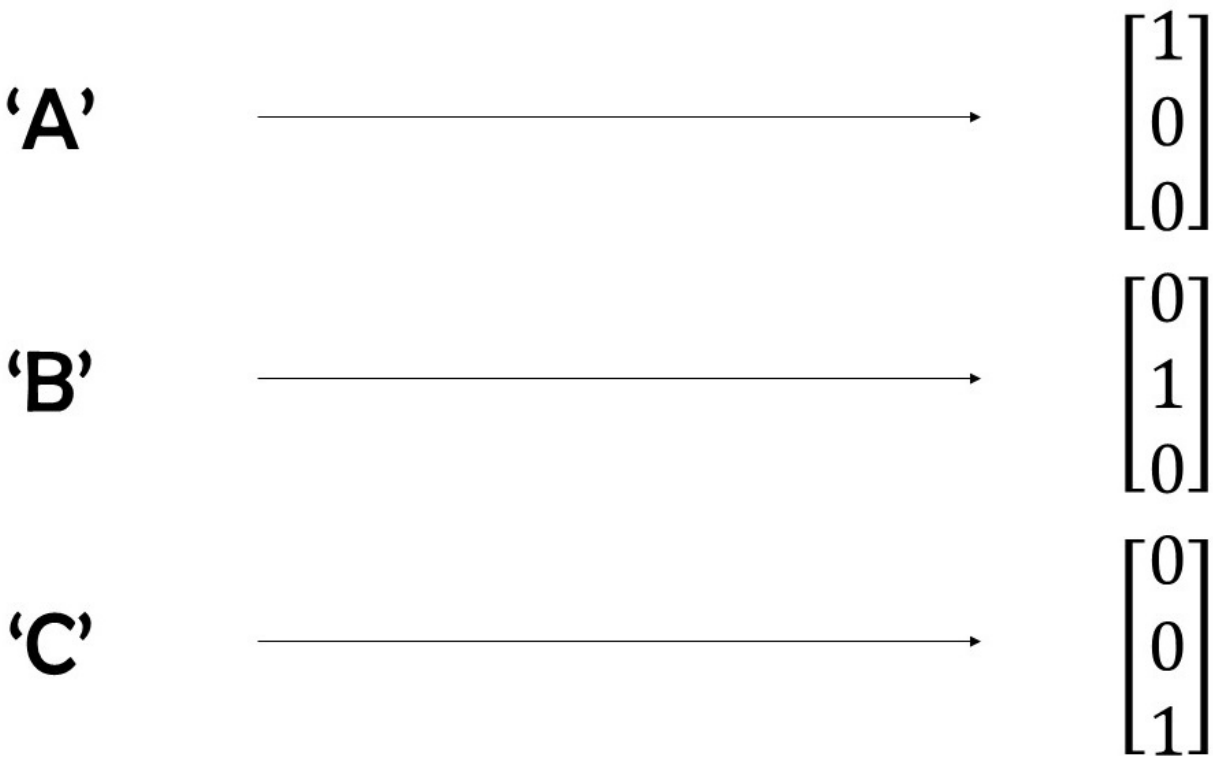
Chapter Goals:

- Process the indicator feature columns used for the machine learning model's input layer

A. One-hot indicators

The remaining non-numeric features in the dataset are the categorical features. Each of these features contains values that can be separated into a fixed number of distinct categories. For example, the `'IsHoliday'` feature contains the categories `0` and `1`, while the `'Type'` feature contains the categories `A`, `B`, and `C`.

These two features specifically (`'IsHoliday'` and `'Type'`) will be the indicator features for the dataset. Since indicator features are categorical, this means they will be represented by one-hot vectors when aggregated into the model's input layer.



Example one-hot vector representation for the 'Type' feature.

B. Categorical column base

When creating the indicator feature columns, we start off with a categorical column base. The categorical column specifies the distinct categories of the feature, as well as the feature's type. We'll use the `tf.feature_column.categorical_column_with_vocabulary_list` list to create the categorical columns.

```
import tensorflow as tf

type_col = tf.feature_column.categorical_column_with_vocabulary_list(
    'Type', ['A', 'B', 'C'], dtype=tf.string)
holiday_col = tf.feature_column.categorical_column_with_vocabulary_list(
    'IsHoliday', [0, 1], dtype=tf.int64)
```

Categorical columns for the 'IsHoliday' and 'Type' features.

We can then convert the categorical column bases into indicator feature columns with the `tf.feature_column.indicator_column` function.

```
import tensorflow as tf

type_feature_col = tf.feature_column.indicator_column(type_col)
holiday_feature_col = tf.feature_column.indicator_column(holiday_col)
```

Converting categorical columns to indicator feature columns.

Time to Code!

In this chapter you'll be creating the indicator feature columns for the dataset by completing the `add_indicator_columns` function. We've already filled the function with skeleton code that iterates through the indicator features in the dataset.

When creating the indicator feature columns, we need to specify the correct datatype. The `'Type'` feature column will have datatype `tf.string`, while the `'IsHoliday'` feature column will have datatype `tf.int64`.

`IsHoliday` feature column will have datatype `tf.int64`.

Set `dtype` equal to the correct datatype, depending on what `feature_name` is.

Each indicator feature column is built from a vocabulary list. The vocabulary list comes from the unique values of the feature in the `final_dataset` DataFrame.

Set `vocab_list` equal to the unique values in `final_dataset[feature_name]`, cast as a list.

Using the vocabulary list and datatype of the feature, we'll create the categorical column for the feature.

Set `vocab_col` equal to `tf.feature_column.categorical_column_with_vocabulary_list` with `feature_name` and `vocab_list` as the required arguments, as well as `dtype` for the `dtype` keyword argument.

After creating the categorical column, we can convert it into the required indicator feature.

Set `feature_col` equal to `tf.feature_column.indicator_column` applied to `vocab_col`. Then append `feature_col` to the end of the `feature_columns` list.

```
import tensorflow as tf

# Add the indicator feature columns to the list of feature columns
def add_indicator_columns(final_dataset, feature_columns):
    indicator_features = ['IsHoliday', 'Type']
    for feature_name in indicator_features:
        # CODE HERE
        pass
```

Embedding Columns

Learn about the embedding feature columns for the ML model's input layer.

Chapter Goals:

- Process the embedding feature columns used for the machine learning model's input layer

A. Feature embeddings

The other feature column for categorical features is embedding feature columns. For our dataset, the embedding features are `'Store'` and `'Dept'`. Both these features have many more categories than the indicator features, so using a one-hot vector representation wouldn't necessarily be the most efficient or useful.

Furthermore, there are connections between categories for both the `'Store'` and `'Dept'` features. For example, we may see similar sales trends between stores in the same region or between departments that sell complementary items (e.g. milk and cereal).

These are trends that can be encapsulated by an embedding feature column, which maps each category to a real-number vector of a specified dimension. We can set the vector dimension to anything, but a good rule of thumb is to set it equal to the 4th root of the number of categories.

```
import tensorflow as tf

stores = list(range(1, 46))
stores_col = tf.feature_column.categorical_column_with_vocabulary_list(
    'StoreID', stores, dtype=tf.int64)
embedding_dim = int(45**0.25)  # 4th root
feature_col = tf.feature_column.embedding_column(
    stores_col, embedding_dim)
```

Creating an embedding feature column for the 'StoreID' feature. There are 45 stores in the dataset.

Creating an embedding feature column for the Store.ID feature. There are 45 stores in the dataset, labeled from 1-45.

For more on word embedding, check out our course Image Recognition on educative.

Time to Code!

In this chapter you'll be creating the embedding feature columns for the dataset by completing the `add_embedding_columns` function. We've already filled the function with skeleton code that iterates through the embedding features in the dataset.

Each embedding feature column is built from a vocabulary list. The vocabulary list comes from the unique values of the feature in the `final_dataset` DataFrame.

Set `vocab_list` equal to the unique values in `final_dataset[feature_name]`, cast as a list.

Using the vocabulary list, we'll create the categorical column for the feature. Each of our dataset's embedding features has datatype `tf.int64`.

Set `vocab_col` equal to `tf.feature_column.categorical_column_with_vocabulary_list` with `feature_name` and `vocab_list` as the required arguments, as well as `tf.int64` for the `dtype` keyword argument.

Following the general rule of thumb, we set the embedding dimension as the 4th root of the size of the vocabulary list.

Set `embedding_dim` equal to the length of `vocab_list` raised to the power of `0.25` and cast as an integer.

```
import tensorflow as tf

# Add the embedding feature columns to the list of feature columns
def add_embedding_columns(final_dataset, feature_columns):
    embedding_features = ['Store', 'Dept']
    for feature_name in embedding_features:
        # CODE HERE
        pass
```

Model Input Layer

Understand the purpose of using feature columns for the model's input layer.

Chapter Goals:

- Aggregate the feature columns for the machine learning model's input layer

A. The input layer

The reason we create feature columns for each of the input data features is so that we can easily make the input layer vector for the machine learning model. The input layer is simply the combination of the numeric, indicator, and embedding feature columns.

Size	CPI	...	Type	...	...	Store	Dept

The input layer vector for the machine learning model. It is the combination of all the feature columns.

Using the functions from the three previous chapters, we can create the feature columns for the entire dataset. The function below, `create_feature_columns` does exactly that.

```
import tensorflow as tf

def create_feature_columns(final_dataset):
    feature_columns = []
    add_numeric_columns(feature_columns)
    add_indicator_columns(final_dataset, feature_columns)
    add_embedding_columns(final_dataset, feature_columns)
    return feature_columns

feature_columns = create_feature_columns(final_dataset)
print(feature_columns)
```

Creating feature columns for the entire dataset. The input final_dataset is the final dataset DataFrame.

In chapter 8 of the **Model Creation** section, you'll use the feature columns to actually create the model's input layer.

Quiz

1

What's the purpose of splitting a dataset into training and evaluation sets?

COMPLETED 0%

1 of 4

Introduction

In this Lab you will be building the machine learning model for predicting sales. Then, after training and evaluation to create the final model, you'll use it to make sales predictions on new data.

A. Which model do we use?

Now that we've analyzed and processed the project's data, we need to choose the correct model for the task. While certain applications benefit from more advanced models (e.g. CNNs for image recognition, LSTMs for NLP), a majority of use cases can be handled by a simple multilayer perceptron.

Our case study deals with predicting weekly sales for various retail stores. The feature data used in the model's input pipeline consists of regular numeric and categorical features, meaning that an MLP will work just fine in making predictions.

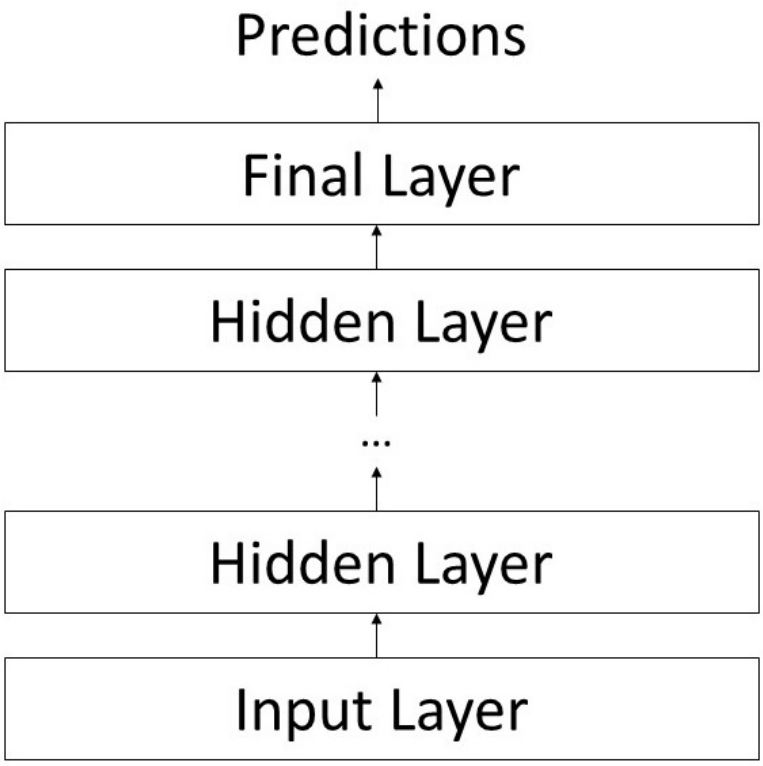
Model Layers

Chapter Goals:

- Create the generalizable function for the MLP model's layers

A. MLP architecture

Our project's model follows the standard MLP architecture. This means that it is made up of multiple fully-connected layers, where each hidden layer uses ReLU activation and the final layer uses no activation. The input layer for the MLP consists of a batch of data observations from the input pipeline (more on this later).



The architecture of the MLP model used for sales predictions.

Larger models (i.e. more hidden layers and nodes) have higher potential to make more accurate predictions, but they can also take longer to train and have a higher chance of overfitting. It's good to experiment with different

have a higher chance of overfitting. It's good to experiment with different model sizes, so we can ultimately choose the best model. This is why we use

an evaluation set in addition to the training set, to compare different model configurations and see which configuration performs best on new data.

For our MLP model, we'll start off with 2 hidden layers. The first hidden layer contains 200 nodes, while the second contains 100. This equates to the list `[200, 100]` for initializing the `SalesModel` class object.

Time to Code!

All code for this chapter goes in the `model_layers` function.

The first layer for the MLP is the input layer. This corresponds to the `inputs` argument of the function.

Set `layer` equal to `inputs`.

The `SalesModel` class is initialized with a `hidden_layers` argument. This is a list of integers, where the integer at index *i* represents the number of nodes in hidden layer *i* of the MLP.

Create a `for` loop that iterates through `self.hidden_layers` using a variable called `num_nodes`.

Each hidden layer of the MLP is a fully-connected layer with ReLU activation, using the previous layer's output as the input.

Inside the `for` loop, set `layer` equal to `tf.layers.dense` applied with `layer` and `num_nodes` as required arguments, along with `tf.nn.relu` as the `activation` keyword argument.

The model's predictions for the input batch of data observations is the output from the MLP's final layer. The final layer has one node, since sales predictions are a single number, and doesn't use any activation function.

Outside the `for` loop, set `batch_predictions` equal to `tf.layers.dense` applied with `layer` and `1` as required arguments.

Return `batch_predictions`.

```
class SalesModel(object):
```

```
def __init__(self, hidden_layers):
  self.hidden_layers = hidden_layers


def model_layers(self, inputs):
  # CODE HERE
```

Regression Function

Chapter Goals:

- Set up the regression function for the model

A. Classification vs. regression

There are two main uses of MLPs in industry: classification and regression. Classification refers to predicting a *class* for a data observation, given its feature data. Examples of classification tasks include:

- Classifying email as real or spam
- Classifying voters as Republican, Democrat, and Independent
- Classifying dolphins into a particular subspecies

The Deep Learning with Tensorflow section of Machine Learning for Software Engineers Lab goes over the details of using MLP models for classification.

The other main usage of MLPs in industry is regression. Regression refers to predicting a real number value for a data observation, given its feature data. Examples of regression tasks include:

- Predicting housing prices based on factors like region and national unemployment rate
- Predicting the weather in Celsius/Fahrenheit
- Predicting the number of points a basketball team will score in a game

For our project, we'll be creating a regression model to predict weekly sales for the departments of each store.

B. Regression loss

For regression models, there are two main loss functions that can used to train the model: mean absolute error and mean squared error. Mean absolute error takes the average absolute difference between the labels (in this case, the

takes the average absolute difference between the labels (in this case, the actual value for the weekly sales) and the model's predicted values. Mean

squared error takes a similar approach, but uses the squared difference rather than the absolute difference.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|label - prediction|$$

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(label - prediction)^2$$

Mean absolute error (MAE) vs. mean squared error (MSE) over a batch of n data observations.

The MSE (also known as L2 loss) amplifies large error values (e.g. a difference of 1000) and minimizes fractional error values (e.g. a difference of 0.01), due to the squaring operation. Since we're predicting sales values that range from about 10,000 to 40,000 dollars, the error values will be in the thousands even when the model is relatively good.

Therefore, we'll use the MAE (also known as L1 loss) to avoid unnecessary error amplification.

C. Input layer

Using the feature columns function from the Data Processing Lab, we can now create the input layer for our model. The model's input layer is just a vector that comes from combining all the numeric, indicator, and embedding feature values in the data.

The `tf.feature_column.input_layer` function allows us to easily convert a dictionary of parsed feature values and a list of feature columns into an input layer for the model.

```
inputs = input_layer(features, cols)
```

Creating the model's input layer from a dictionary of feature values (features) and list of feature columns (cols). The tf.feature_column prefix is omitted from the input_layer function for brevity.

Time to Code!

All code for this chapter goes in the `regression_fn` function.

Using the `create_feature_columns` function from the Data Processing Lab, we'll create the input layer's feature columns.

Set `feature_columns` equal to `create_feature_columns` applied with no arguments.

Using the feature columns and feature data (i.e. the `features` input variable), we can create the model's input layer.

Set `inputs` equal to `tf.feature_column.input_layer` applied with `features` and `feature_columns` as required arguments.

Using the `model_layers` function completed in the previous chapter, we'll obtain the batch predictions for the input layer.

Set `batch_predictions` equal to `self.model_layers` applied to `inputs`.

The `batch_predictions` tensor has shape `(batch_size, 1)`, while the `labels` input argument has shape `(batch_size,)`. To calculate the model's loss, we need the two tensors to have the same shape.

We can use `tf.squeeze` on `batch_predictions` to remove the extraneous dimension of size `1`. This results in the same shape as `labels`.

Set `predictions` equal to `tf.squeeze` applied with `batch_predictions` as the only argument.

We calculate the model's loss during training and evaluation, i.e. when there are labels for the input data. The loss function will be a simple mean absolute error.

Create an `if` block that checks if `labels` is not `None`.

We'll use the `tf.losses.absolute_difference` function to calculate the mean absolute error between `labels` and `predictions`.

Inside the `if` block, set `loss` equal to `tf.losses.absolute_difference` applied with `labels` and `predictions` as required arguments.

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def regression_fn(self, features, labels, mode, params):
    # CODE HERE
```

```
  # CODE HERE

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Training Mode

Chapter Goals:

- Set up the regression function's training code

A. The `EstimatorSpec`

There are three phases to completing the machine learning model: training, evaluation, and prediction. With TensorFlow, we can easily bundle the three phases into a single function using `EstimatorSpec` objects for each phase.

The `EstimatorSpec` object has three modes corresponding to the three phases:

- Training: `tf.estimator.ModeKeys.TRAIN`
- Evaluation: `tf.estimator.ModeKeys.EVAL`
- Prediction: `tf.estimator.ModeKeys.PREDICT`

The `EstimatorSpec` is initialized with whatever mode corresponds to the current phase of completing the machine learning model. For training, the `EstimatorSpec` is also initialized with the model's loss and training operation (for minimizing the loss).

```
inputs = input_layer(features, cols)
```

Creating the model's input layer from a dictionary of feature values (features) and list of feature columns (cols). The tf.feature_column prefix is omitted from the input_layer function for brevity.

Time to Code!

All code for this chapter goes in the `regression_fn` function.

The `mode` argument for `regression_fn` lets us know whether the model is currently being run for training, evaluation, or prediction. The code for this chapter focuses on model training.

Outside the `if` block from the previous chapter, create another `if` block. This one should check if `mode` is equal to `tf.estimator.ModeKeys.TRAIN`.

The rest of the code for this chapter goes inside the `if` block.

To keep track of the total number of training steps taken during multiple different training runs, we'll set up a global step object.

Set `global_step` equal to `tf.train.get_or_create_global_step` applied with no arguments.

We'll minimize the model's loss during training using the ADAM optimization method, via the `AdamOptimizer` object.

Set `adam` equal to `tf.train.AdamOptimizer` initialized with no arguments.

Set `train_op` equal to `adam.minimize` applied with `loss` as the required argument and `global_step` as the `global_step` keyword argument.

We can now create and return the `EstimatorSpec` object for model training.

Return `tf.estimator.EstimatorSpec` initialized with `mode` as the required argument and `loss` and `train_op` as the `loss` and `train_op` keyword arguments.

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)
    # CODE HERE


  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Evaluation Mode

Chapter Goals:

- Set up the regression function's evaluation code

A. Evaluating the model

When evaluating the model, we use mean absolute error as the metric. This is because our goal is to get the model's sales predictions as close to the actual labels as possible, which is equivalent to minimizing the mean absolute error between predictions and labels.

Since we use the same evaluation metric as the loss function, this makes the evaluation code extremely easy. We just need to return an `EstimatorSpec` containing the model's loss on the evalution set.

```
mode = tf.estimator.ModeKeys.TRAIN
estimator_spec = tf.estimator.EstimatorSpec(
    mode, loss=loss, train_op=train_op)
```

Creating the EstimatorSpec for training. The train_op variable is used for minimizing the loss

Time to Code!

All code for this chapter goes in the `regression_fn` function.

The code for this chapter focuses on model evaluation. Since we evaluate the model using mean absolute error, which is the same as the loss function, we don't need to do anything other than return the `EstimatorSpec`.

Outside the `if` block from the previous chapter, create another `if` block. This one should check if `mode` is equal to `tf.estimator.ModeKeys.EVAL`.

Inside the `if` block, return `tf.estimator.EstimatorSpec` initialized with `mode` as the required argument and `loss` as the `loss` keyword argument.

```
class SalesModel(object):
  def __init__(self, hidden_layers):

    self.hidden_layers = hidden_layers

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)

    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    # CODE HERE

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Prediction Mode

Chapter Goals:

- Set up the regression function's prediction code

A. Prediction values

For the prediction mode in the regression function, we initialize and return an `EstimatorSpec` object containing a dictionary with the model's predictions. The model's predictions need to be in 2-D tensor format, with shape `(batch_size, 1)`.

Using the 1-D tensor version (which was used in calculating the loss) will result in an indexing error when making predictions on a TFRecords dataset.

```
mode = tf.estimator.ModeKeys.PREDICT
prediction_info = { 'predictions': batch_predictions }
estimator_spec = tf.estimator.EstimatorSpec(
    mode, predictions=prediction_info)
```

Creating the EstimatorSpec for predictions

Time to Code!

All code for this chapter goes in the `regression_fn` function.

The code for this chapter focuses on model predictions. The `predictions` variable used in calculating the loss is a 1-D tensor, which can cause indexing issues if we use it when running predictions on a TFRecords dataset.

Instead, we'll use `batch_predictions` (which has shape `(batch_size, 1)` in the predictions dictionary.

Outside the `if` block from the previous chapter, create another `if` block. This one should check if `mode` is equal to `tf.estimator.ModeKeys.PREDICT`.

Inside the `if` block, create a dictionary called `prediction_info`, which

Inside the `if` block, create a dictionary called `prediction_info`, which maps the string `'predictions'` to the tensor `batch_predictions`.

We can now return an `EstimatorSpec` object for the predictions.

Inside the `if` block, return `tf.estimator.EstimatorSpec` initialized with `mode` as the required argument and `prediction_info` as the `predictions` keyword argument.

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)

    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    if mode == tf.estimator.ModeKeys.EVAL:
      return tf.estimator.EstimatorSpec(mode, loss=loss)
    # CODE HERE


  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Regression Model

Chapter Goals:

- Create the `Estimator` object for the regression model

A. `Estimator` object

The entire regression model, from training to evaluation to predictions, can be encapsulated in a single `Estimator` object. The `Estimator` object is initialized with the regression function, as well as a few keyword arguments.

One of the keyword arguments is `model_dir`, which represents the path to the directory that will contain the model's checkpoints. The checkpoints are how we save and restore the model's parameters for training, evaluation, and making predictions.

Another keyword argument we'll use is `config`, which specifies a custom configuration for the model. We can customize values like logging frequency, the maximum number of checkpoints to keep, and how often we save a summary of the training. These can all be set by creating a `RunConfig` object with the designated values.

For our regression model, the only custom configuration we'll set is the logging frequency. This refers to how frequently the model will log the loss and global step values to the screen during training. The default frequency is every 100 training steps, but since our model's training steps are very quick, we'll set the frequency to every 5000 steps.

```
config = tf.estimator.RunConfig(log_step_count_steps=5000)
regression_model = tf.estimator.Estimator(
    regression_fn,  # the model's regression function
    config=config,
    model_dir='model_ckpts')
```

Creating the Estimator object for our regression model. We set the checkpoint directory to

'model_ckpts', and set the logging frequency to every 5000 steps.

Time to Code!

All code for this chapter goes in the `create_regression_model` function.

The default configuration for training logs the loss and global step values every 100 training steps. However, since each step is very quick when training our regression model, we'll instead choose to log the values every 5000 steps.

Set `config` equal to `tf.estimator.RunConfig` initialized with `log_step_count_steps` set to `5000`.

The regression model will be an `Estimator` object, which uses `regression_fn` as the model function and `ckpt_dir` as the path to the model's checkpoints.

We'll also use the `config` variable from the previous step to ensure logging every 5000 steps during training.

Set `regression_model` equal to `tf.estimator.Estimator` initialized with `self.regression_fn` as the required argument, along with `config` and `ckpt_dir` for the `config` and `model_dir` keyword arguments.

Return `regression_model`.

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def create_regression_model(self, ckpt_dir):
    # CODE HERE

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)

    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    if mode == tf.estimator.ModeKeys.EVAL:
      return tf.estimator.EstimatorSpec(mode, loss=loss)
    if mode == tf.estimator.ModeKeys.PREDICT:
      prediction_info = {
```

```
        'predictions': batch_predictions
    }
    return tf.estimator.EstimatorSpec(mode, predictions=prediction_info)

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Model Training

Chapter Goals:

- Train the regression model

A. Training with the `Estimator`

Since we set up all the code for the regression model, we can now train the model using the *train.tfrecords* file we created in the Data Processing Lab. The `Estimator` object contains a `train` function that lets us easily train the model.

Interestingly, the `train` function's only required argument is a function that takes in no input arguments. This function should set up the input pipeline for the model training.

In our case, it will return the training dataset using the `create_tensorflow_dataset` function from the Data Processing Lab

```
input_fn = lambda:create_tensorflow_dataset('train.tfrecords', 50)
regression_model.train(input_fn)
```

Using the Estimator object (regression_model) to run model training. We train with a batch size of 50 and save checkpoints to the model_ckpts directory.

Since the `create_tensorflow_dataset` function repeats the dataset indefinitely for training, the above code runs training until we manually stop the process. However, if we want to run training for a fixed number of steps, we can set the `steps` keyword argument.

```
regression_model.train(input_fn, steps=20000)
```

Training for 20000 steps.

Note that the default value for `steps` is `None`, which signifies that training will run until the end of the input dataset. If the input dataset is repeated

indefinitely (as is the case in our training), setting `steps` to `None` will run training until it is manually terminated.

Code for training the regression model is shown below.

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def run_regression_training(self, ckpt_dir, batch_size, num_training_steps=None):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset('train.tfrecords', batch_size)
    regression_model.train(input_fn, steps=num_training_steps)

  def create_regression_model(self, ckpt_dir):
    config = tf.estimator.RunConfig(log_step_count_steps=5000)
    regression_model = tf.estimator.Estimator(
      self.regression_fn,
      config=config,
      model_dir=ckpt_dir)
    return regression_model

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)

    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    if mode == tf.estimator.ModeKeys.EVAL:
      return tf.estimator.EstimatorSpec(mode, loss=loss)
    if mode == tf.estimator.ModeKeys.PREDICT:
      prediction_info = {
          'predictions': batch_predictions
      }
      return tf.estimator.EstimatorSpec(mode, predictions=prediction_info)

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

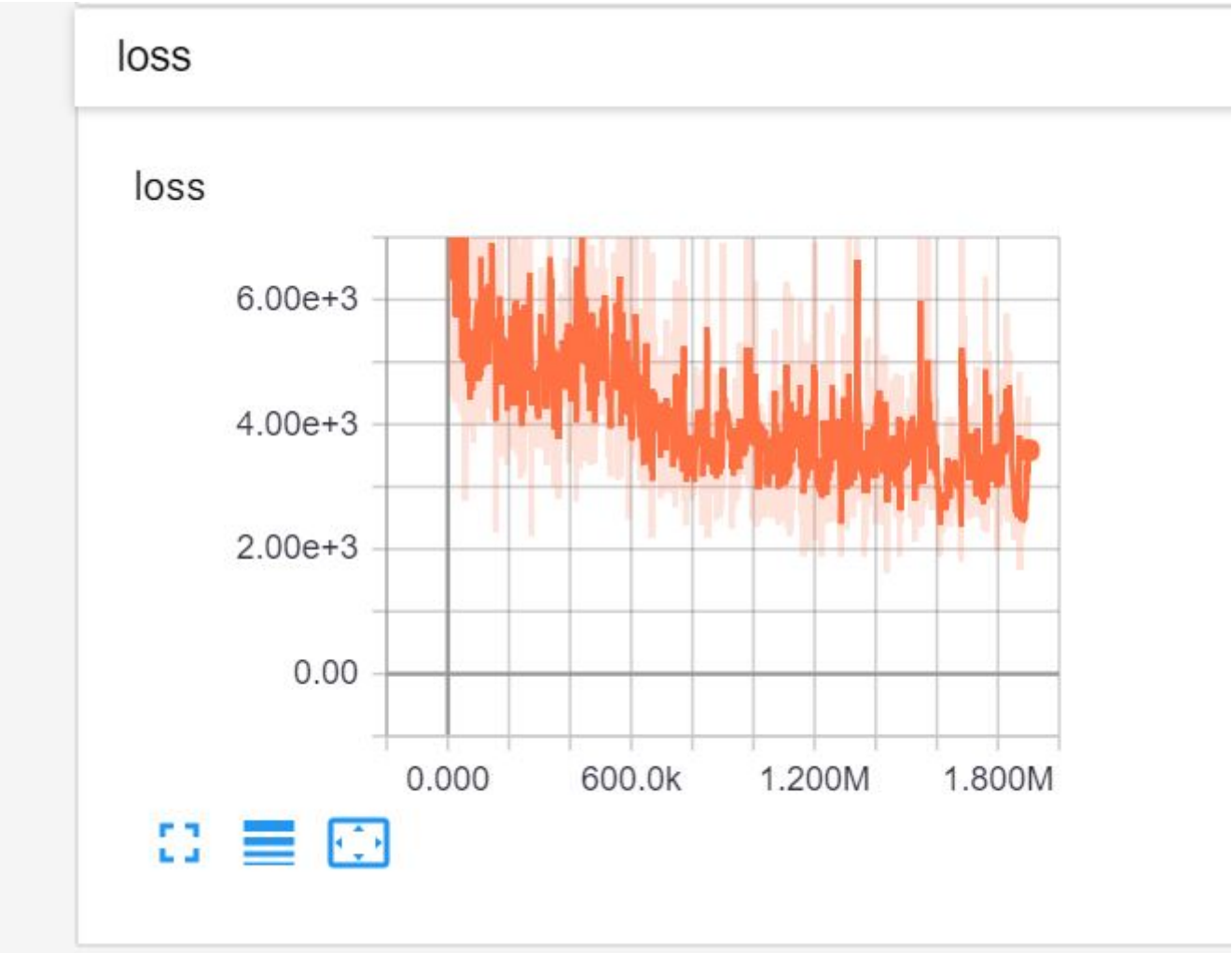
Model Evaluation

Chapter Goals:

- Evaluate the regression model

A. Evaluating with the `Estimator`

We train the model long enough that the loss begins to show signs of convergence (for our 2 hidden layer MLP model, this is around 2M training steps). To get a better look at how the loss is progressing, we can take a look at the TensorBoard training visualizations (see the Model Execution section of Deep Learning for Industry for more details on TensorBoard).



The TensorBoard line plot showing the loss progression.

We evaluate with the `Estimator` in almost the exact same way we train, with the main difference being that we use the `evaluate` function rather than the `train` function.

The evaluation dataset is contained in the *eval.tfrecords* file (created in the Data Processing Lab). The batch size argument for creating the evalution TFRecords dataset only affects evaluation speed. A larger batch size can provide a speedup in evaluation, although you have to make sure the batch is small enough to be contained in memory.

We'll use a batch size of 50 in our evaluation, which is definitely small enough to fit in memory and also faster than using a batch size of 1. The entire evaluation process takes about a minute.

```
input_fn = lambda:create_tensorflow_dataset(
    'eval.tfrecords', 50, training=False)
regression_model.evaluate(input_fn)
```

Using the Estimator object (regression_model) to evaluate the model trained with 1.9M steps. The loss (MAE) on the evaluation set is about 3267.

Code for evaluating the regression model is shown below

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def run_regression_eval(self, ckpt_dir):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset('eval.tfrecords', 50, training=False)
    return regression_model.evaluate(input_fn)

  def run_regression_training(self, ckpt_dir, batch_size, num_training_steps=None):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset('train.tfrecords', batch_size)
    regression_model.train(input_fn, steps=num_training_steps)

  def create_regression_model(self, ckpt_dir):
    config = tf.estimator.RunConfig(log_step_count_steps=5000)
    regression_model = tf.estimator.Estimator(
      self.regression_fn,
      config=config,
      model_dir=ckpt_dir)
    return regression_model

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)
```

```
    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)


    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    if mode == tf.estimator.ModeKeys.EVAL:
      return tf.estimator.EstimatorSpec(mode, loss=loss)
    if mode == tf.estimator.ModeKeys.PREDICT:
      prediction_info = {
          'predictions': batch_predictions
      }
      return tf.estimator.EstimatorSpec(mode, predictions=prediction_info)

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Making Predictions

Chapter Goals:

- Use the regression model to make predictions on an unlabeled test dataset

A. The final model

After continuous training and evaluation of the sales prediction model, we found that the initial configuration of the model (2 hidden layers with 200 and 100 nodes, respectively) performs reasonably better than smaller models and has essentially the same performance as larger models. Therefore, we stuck with the initial configuration for the final model

The final model was trained with 2.5M training steps, which took about 8 hours using a single NVIDIA Tesla P100 GPU. The final model's evaluation loss was 2873.11, which is pretty good considering the large range of possible sales values (up to $70,000).

B. `Estimator` predictions

After notifying the project's supervisor that the model is ready, she hands us the unlabeled testing data in a CSV file. The testing data has the exact same format as the training data, minus the `'Sales'` feature. Our job is to make sales predictions for each of the observations.

Just like in training and evaluation, we make predictions with the `Estimator` by using a lambda wrapper for the input pipeline's dataset. In this case, the input pipeline contains all the unlabeled data in a TFRecords dataset.

Using the code from the Data Analysis and Data Processing Labs on the testing data's CSV file, we store the CSV data into a TFRecords file called *test.tfrecords*. We then use this file for the predictions input pipeline.

```
input_fn = lambda:create_tensorflow_dataset(
    'test.tfrecords', 1, training=False, has_labels=False)
predictions = regression_model.predict(input_fn)
```

Using the `Estimator` object (`regression_model`) to make predictions on the test dataset. The predictions are stored in a generator (`predictions`).

Using the `Estimator` object's `predict` function, we make predictions on the unlabeled dataset one observation at a time (i.e. a batch size of 1). The `predict` function returns a generator object, which we convert into a list of the prediction values (see the **Code** tab for details).

Since we don't shuffle the test dataset, the model's predictions can be easily mapped back to their corresponding data observation. In other words, the value at index \(\small i\) of the final prediction list is the sales prediction for the \(\small i^{th}\) observation in the test set (the \(\small i^{th}\) row of the CSV file).

Code for using the model to make predictions is shown below

```
class SalesModel(object):
  def __init__(self, hidden_layers):
    self.hidden_layers = hidden_layers

  def run_regression_predict(self, ckpt_dir, data_file):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset(data_file, 1, training=False, has_labels=Fals
    predictions = regression_model.predict(input_fn)
    pred_list = []
    for pred_dict in predictions:
        pred_list.append(pred_dict['predictions'][0])
    return pred_list

  def run_regression_eval(self, ckpt_dir):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset('eval.tfrecords', 50, training=False)
    return regression_model.evaluate(input_fn)

  def run_regression_training(self, ckpt_dir, batch_size, num_training_steps=None):
    regression_model = self.create_regression_model(ckpt_dir)
    input_fn = lambda:create_tensorflow_dataset('train.tfrecords', batch_size)
    regression_model.train(input_fn, steps=num_training_steps)

  def create_regression_model(self, ckpt_dir):
    config = tf.estimator.RunConfig(log_step_count_steps=5000)
    regression_model = tf.estimator.Estimator(
      self.regression_fn,
      config=config,
      model_dir=ckpt_dir)
    return regression_model

  def regression_fn(self, features, labels, mode, params):
    feature_columns = create_feature_columns()
```

```
    inputs = tf.feature_column.input_layer(features, feature_columns)
    batch_predictions = self.model_layers(inputs)
    predictions = tf.squeeze(batch_predictions)

    if labels is not None:
      loss = tf.losses.absolute_difference(labels, predictions)

    if mode == tf.estimator.ModeKeys.TRAIN:
      global_step = tf.train.get_or_create_global_step()
      adam = tf.train.AdamOptimizer()
      train_op = adam.minimize(
        loss, global_step=global_step)
      return tf.estimator.EstimatorSpec(mode, loss=loss, train_op=train_op)
    if mode == tf.estimator.ModeKeys.EVAL:
      return tf.estimator.EstimatorSpec(mode, loss=loss)
    if mode == tf.estimator.ModeKeys.PREDICT:
      prediction_info = {
          'predictions': batch_predictions
      }
      return tf.estimator.EstimatorSpec(mode, predictions=prediction_info)

  def model_layers(self, inputs):
    layer = inputs
    for num_nodes in self.hidden_layers:
      layer = tf.layers.dense(layer, num_nodes,
        activation=tf.nn.relu)
    batch_predictions = tf.layers.dense(layer, 1)
    return batch_predictions
```

Final Report

Chapter Goals:

- Learn what goes inside the project's final report, and how it should be presented

A. Creating the final report

After making the predictions on the testing dataset, we can create the final report for the project. The report contains all major parts of the project, including data analysis, model overview, and results from training, evaluation, and predictions. The best way to present the report is through a slideshow presentation, which allows us to convey results with meaningful visualizations (e.g. plots, charts, etc.). Each slide should have limited text.

The project report should focus more on the data and results, rather than the machine learning model. It's good to have an overview of the model's architecture, but avoid going too in-depth into how the model actually works. A short description of the model and diagram of the architecture should be enough.

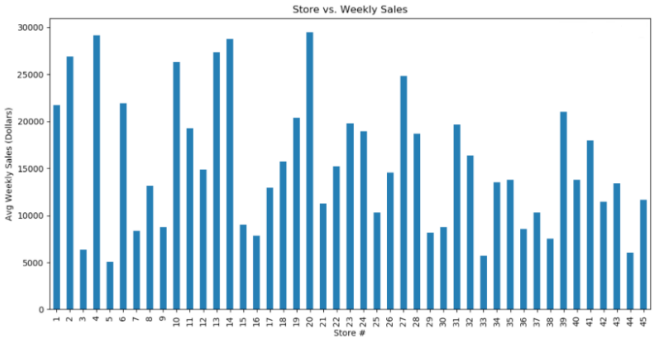
To see what this project's report looks like, check out the presentation in the right-hand column of the screen.

The Problem

- Sales forecasting: predicting weekly sales amounts
- Different departments across 45 stores
- Can we train a machine learning model to automatically make these sales predictions?

The Dataset

- Each data observation describes sales for a department of one store in a given week
- Helpful features in the dataset include store sizes, type of store, regional temperature, and national unemployment rate



The Model

- Converted CSV data into TFRecords pipeline
- Trained a 2 hidden layer multilayer perceptron (MLP)

Results

- Used mean absolute error (MAE) as training/evaluation metric
- Final evaluation MAE: 2873.11 (avg. $2873 prediction error on evaluation set)
- Weekly sales can range up to $70,000

SAMPLE

Store ID: 32
Department ID: 56
Start of Week: 4/12/2019
Holiday Week: No
Regional Temperature: 59.4 °F
Regional Fuel Price: $3.71/gallon
CPI (Consumer Price Index): 223.03
Unemployment %: 6.21
Store Type: A
Store Size: 203,000 square feet
Predicted Sales: $21,206.14

Course Conclusion

A. Closing Thoughts

Machine Learning (ML) is one of the hottest fields in technology right now, and is inherently relevant to artificial intelligence and data science. From healthcare and agriculture to manufacturing and retail, many companies across a variety of different industries are leveraging these technologies to get ahead.

B. Course recap

This course has provided an overview of how to write useful code and impactful machine learning applications. You'll be able to take the practical lessons and actionable insights from this course and apply them to your projects.

C. Topics covered

- Data analysis/visualization
- Feature engineering
- Data pipeline
- Supervised learning.

D. Industry standard frameworks used in this course

- NumPy
- pandas
- TensorFlow